

RELAZIONE TECNICA DI PROGETTO

Corso: Web and Multimedia Technologies (Laurea Magistrale) **Docente:** Prof. Marco Porta — Università degli Studi di Pavia **Studente:** Marco Santoro **Progetto:** Algora Studio — sito istituzionale e piattaforma di contatto **Anno Accademico:** 2025/2026

1. INTRODUZIONE E CONTESTO APPLICATIVO

Questa relazione descrive la progettazione e le scelte tecniche del sito web di **Algora Studio**, uno studio di sviluppo software specializzato in soluzioni verticali per il patrimonio culturale e la memoria storica documentale italiana (Archivi di Stato, archivi storici comunali, studi notarili, fondazioni).

Il prodotto attorno a cui ruota la comunicazione del sito è **Foliarium**, un gestionale sviluppato per l'Archivio di Stato di Savona per la digitalizzazione, la modellazione relazionale e la consultazione fuzzy degli archivi catastali storici (dal 1830 a oggi).

Motivazione della scelta

La maggior parte dei siti per software house si appoggia a template generici o a CMS che frammentano il controllo sul codice. Realizzare un portale ad hoc risponde all'obiettivo di dimostrare che la programmazione web nativa (**HTML5**, **CSS3**, **JavaScript Vanilla**, **PHP** e **MySQL/MariaDB**) permette di ottenere:

1. un'estetica caratterizzata ("Archivio Caldo") coerente con il dominio trattato;
 2. un carico di rete ridotto, con un solo foglio di stile e un solo file JavaScript;
 3. conformità alla validazione W3C e alle regole di base di usabilità e accessibilità viste a lezione.
-

2. ARCHITETTURA DELL'INFORMAZIONE E MAPPATURA DEI FILE

Il sito si articola su **5 pagine di contenuto**, servite da PHP, più **1 script server-side** per la persistenza dei dati. Le parti comuni sono estratte in due **include PHP** riusati da tutte le pagine.

File	Ruolo
index.php	Home. Vision dello studio, manifesto, metriche di sintesi, anteprima del caso studio, sezione approccio, call to action finale.
chi-siamo.php	Profilo dello studio. Filosofia dello sviluppo verticale, i tre valori guida, profilo del fondatore, stack tecnologico, prospettive future.
foliarium.php	Scheda prodotto. Funzionalità di Foliarium (ricerca fuzzy, albero delle proprietà, audit trail, esportazione report), requisiti di sistema, licenze e pacchetti di assistenza.

File	Ruolo
caso-studio.php	Caso studio Archivio di Stato di Savona. Risultati quantitativi (69 comuni, 12.000+ partite, 8.500+ possessori), confronto “prima e dopo”, fasi del progetto.
contatti.php	Contatti e demo. Guida in quattro fasi, modulo di contatto, FAQ a fisarmonica.
nav.php	Include: barra di navigazione, generata da un array PHP che marca automaticamente la voce attiva.
footer.php	Include: piè di pagina comune a tutte le pagine.
invia-contatto.php	Backend. Riceve il POST del modulo, valida, inserisce nel database con istruzione preparata, restituisce la pagina di esito.
config-db.php	Parametri di connessione al database, isolati dalla logica applicativa.
crea_db.sql	Script DDL di creazione di database e tabella.
style.css	Foglio di stile unico dell'intero sito (1.179 righe, ~44 KB).
main.js	Comportamenti client-side (53 righe).

2.1 Perché PHP anche sulle pagine di contenuto

Le cinque pagine non contengono logica applicativa, ma hanno estensione .php per poter usare include:

```
<?php $active = 'home'; ?>
...
<?php include 'nav.php'; ?>
```

Barra di navigazione e piè di pagina esistono così in **un'unica copia**. La voce di menu corrispondente alla pagina corrente viene evidenziata confrontando la variabile \$active con le chiavi dell'array che descrive il menu:

```
$navItems = [
    'home'      => ['href' => 'index.php',    'label' => 'Home'],
    'chi-siamo' => ['href' => 'chi-siamo.php', 'label' => 'Chi siamo'],
    /* ... */
];
foreach ($navItems as $key => $item) {
    /* class="active" quando $active === $key */
}
```

Senza include, una modifica al menu andrebbe replicata a mano su sei file, con il rischio concreto di disallineamenti.

3. FRONTEND: HTML5 SEMANTICO, CSS3 E DESIGN SYSTEM

3.1 Scrittura nativa e struttura semantica

Il codice è scritto interamente a mano, senza editor WYSIWYG e senza framework CSS (Bootstrap, Tailwind). Ogni pagina usa i tag strutturali di HTML5: `<nav>` per la navigazione, `<main>` per il contenuto principale, `<header>` per le sezioni di apertura, `<section>` per i blocchi tematici, `<footer>` per la chiusura.

L'intero contenuto di ogni pagina è racchiuso in `<main id="contenuto">`: questo definisce il landmark principale e permette il funzionamento del collegamento di salto descritto in §6.

3.2 Design system “Archivio Caldo”

La palette e la tipografia sono definite con **variabili CSS** (custom properties) raccolte in `:root`, così che un cambio di tonalità si propaghi a tutto il sito modificando una riga sola:

- **Colori:** fondo pergamena (`--parchment: #F4EFE4`), `--cream: #F9F6EF`), testo e contenitori inchiostro (`--ink: #1E150A`), accenti dorati (`--gold: #B8821A`), verde archivio per le conferme (`--green: #1A3A28`).
- **Tipografia:** Cormorant Garamond per i titoli display e Libre Baskerville per il testo corrente, entrambi da Google Fonts; Trebuchet MS / Calibri in maiuscolo spaziato per etichette e micro-testi.

I due caratteri sono l'unica dipendenza esterna del progetto, caricata via `@import` in testa a `style.css`. In un contesto di produzione converrebbe ospitarli localmente, sia per eliminare la richiesta a un dominio terzo sia per non far dipendere il rendering da un servizio esterno; per il perimetro di questo progetto la dipendenza è stata mantenuta e qui dichiarata esplicitamente.

Come descritto in §6.2, l'oro di marca è usato per filetti, bordi e sfondi, mentre **come colore di testo** il foglio di stile ricorre a due varianti calibrate sul fondo, perché la tonalità originale non raggiungeva il contrasto minimo richiesto.

3.3 Layout non lineare

Il requisito di un layout non semplicemente lineare è soddisfatto da tre tecniche, tutte fuori dal flusso normale del documento:

1. **Barra di navigazione ancorata:** `#nav` usa `position: fixed; top: 0; left: 0; right: 0; z-index: 200`, restando visibile durante lo scorrimento.
2. **Griglie asimmetriche con CSS Grid:** la hero e il caso studio usano colonne di larghezza diversa (`grid-template-columns: 1fr 400px`) e griglie a gap: `1px` su fondo colorato, che producono l'effetto di “caselle” archivistiche separate da un filetto. In tutto il foglio di stile sono presenti 18 contesti di griglia.
3. **Confronto “prima e dopo”:** in `caso-studio.php`, due colonne a contrasto invertito (pergamena contro inchiostro) affiancano il processo manuale e quello digitalizzato.

A questi si aggiungono elementi in `position: absolute` usati come decorazione (la lettera “A” in filigrana nella hero, i punti della timeline).

3.4 Comportamento responsive: impostazione mobile-first

Il foglio di stile è scritto **mobile-first**: le regole di base descrivono lo schermo stretto, e un unico punto di rottura a 961px introduce la disposizione desktop tramite @media (min-width: 961px).

Concretamente, ogni griglia nasce a colonna singola:

```
.feat-grid {  
  display: grid; grid-template-columns: 1fr;  
  gap: 24px; background: var(--border);  
}
```

e le colonne multiple arrivano dal blocco desktop della sezione:

```
@media (min-width: 961px) {  
  .feat-grid { grid-template-columns: 1fr 1fr; gap: 2px; }  
}
```

I blocchi @media non sono raccolti in coda al file ma collocati **al termine della sezione che riguardano** — uno per le griglie comuni, uno per il piè di pagina, uno per ciascuna pagina — così che la variante desktop di un componente si legga accanto alla sua definizione di base.

Il passo orizzontale delle fasce a tutta larghezza è centralizzato in una variabile, ridichiarata una sola volta per il desktop:

```
:root { --pad-x: 20px; }  
@media (min-width: 961px) {  
  :root { --pad-x: clamp(24px, 5vw, 72px); }  
}
```

Undici regole (barra di navigazione, sezioni, fasce, piè di pagina, intestazioni di pagina) usano var(--pad-x): il margine di pagina si modifica in un punto solo.

Perché non uno strato correttivo max-width L'impostazione precedente definiva il layout desktop nelle regole di base e lo correggeva a valle con un blocco @media (max-width: 960px) che riportava a colonna singola tutte le griglie. Quel blocco aveva bisogno di **nove dichiarazioni !important** per vincere sulle regole che stava annullando, ed è un difetto strutturale, non estetico: !important scavalca la specificità, quindi una regola mirata perde contro una regola generica dichiarata dopo di essa.

Il caso concreto emerso in questo progetto: i titoli di colonna del piè di pagina sono governati da .footer-col h2 { font-size: 10px }, ma lo strato correttivo conteneva h2 { font-size: clamp(28px, 6vw, 36px) !important }. Su schermo stretto vinceva quest'ultima, e le etichette "Studio", "Prodotti", "Contatti" venivano rese a 28px anziché a 10px — visibile solo sotto i 960px. Rimosso lo strato, la regola specifica torna a valere e il difetto sparisce senza alcun intervento mirato.

Nel foglio di stile non resta oggi nessun !important di layout: gli unici tre superstiti sono quelli, idiomatici, del blocco prefers-reduced-motion, dove servono proprio a scavalcare qualunque animazione dichiarata altrove.

4. CLIENT-SIDE: JAVASCRIPT VANILLA

main.js contiene tre comportamenti, scritti in JavaScript ES6+ senza librerie.

- **Menu di navigazione (mobile).** Il pulsante apre e chiude l'elenco dei collegamenti, aggiornando l'attributo `aria-expanded`; il tasto Esc chiude il menu e riporta il focus sul pulsante che lo aveva aperto.
- **Ombra della barra allo scorrimento.** Un listener su `scroll` aggiunge la classe `.scrolled` a `#nav` oltre i 20 px, staccando visivamente la barra dal contenuto.
- **FAQ a fisarmonica.** Al clic su una domanda le altre risposte si chiudono; l'apertura è animata sulla proprietà `max-height` e comunicata alle tecnologie assistive tramite `aria-expanded`.

Entrambi i comandi interattivi sono elementi **<button> nativi**: la scelta è discussa in §6.1.

5. SERVER-SIDE E PERSISTENZA DATI

5.1 Architettura

Un modulo che invii una email via `mailto:` o simuli l'invio in JavaScript non garantisce né persistenza né tracciabilità. Il progetto adotta quindi l'architettura a tre livelli **Client (HTML/CSS/JS) → Application server (PHP 8) → Database server (MySQL/MariaDB)**.

5.2 Base di dati (algora_db)

```
CREATE DATABASE IF NOT EXISTS algora_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE algora_db;

CREATE TABLE IF NOT EXISTS contatti (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nome        VARCHAR(100) NOT NULL,
  ente        VARCHAR(100),
  email       VARCHAR(100) NOT NULL,
  tipo        VARCHAR(50),
  messaggio   TEXT          NOT NULL,
  data_invio  DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

La codifica `utf8mb4` copre l'intero repertorio Unicode, inclusi i caratteri accentati e i segni tipografici usati nei nomi di enti e località.

5.3 Flusso di elaborazione

1. **Verifica del metodo HTTP.** Una richiesta che non sia POST (tipicamente l'apertura diretta dell'URL) non ha dati da elaborare: lo script risponde con un redirect 303 See Other verso `contatti.php`, invece di mostrare una pagina di errore priva di significato.
2. **Validazione.** `trim()` sui campi, controllo dei campi obbligatori, `filter_var($email, FILTER_VALIDATE_EMAIL)` per la correttezza formale dell'indirizzo. Messaggi di errore distinti per campi mancanti e per email non valida.
3. **Inserimento con istruzione preparata.**

```
$stmt = $conn->prepare(
    'INSERT INTO contatti (nome, ente, email, tipo, messaggio)
    VALUES (?, ?, ?, ?, ?)'
);
$stmt->bind_param('sssss', $nome, $ente, $email, $tipo, $messaggio);
$stmt->execute();
```

I segnaposto ? e il binding separato dei parametri tengono distinti comandi SQL e dati forniti dall'utente: il contenuto dei campi non può essere reinterpretato come istruzione. Un invio con nome = "Rossi'); DROP TABLE contatti; --" viene memorizzato come stringa letterale e la tabella resta intatta. 4. **Pagina di esito.** In base all'esito lo script produce una pagina di conferma personalizzata o una scheda di errore, entrambe con la navigazione, il piè di pagina e il foglio di stile del resto del sito.

5.4 Escaping: una sola volta, in uscita

I dati vengono salvati nel database **così come l'utente li ha scritti**, e `htmlspecialchars()` viene applicato soltanto al momento di stamparli nella pagina.

Applicare l'escaping anche in ingresso — errore frequente, perché sembra “più sicuro” — produce una doppia codifica: un cognome come Sant 'Angelo verrebbe memorizzato come Sant'Angelo e ristampato a video come Sant&#039;Angelo. Il database si riempirebbe di entità HTML al posto dei caratteri reali, rendendo inutilizzabili i dati per qualsiasi uso diverso dalla pagina web (esportazioni, email, ricerche). La protezione dalle SQL injection è affidata all'istruzione preparata, non all'escaping HTML, che serve a un problema diverso (Cross-Site Scripting, in uscita).

5.5 Gestione degli errori e credenziali

I parametri di connessione stanno in `config-db.php`, separati dalla logica: il passaggio da ambiente locale a hosting richiede di modificare un solo file, e in un deployment reale il file va escluso dal versionamento.

A partire da PHP 8.1 l'estensione `mysqli` segnala gli errori **sollevando eccezioni** anziché valorizzando `connect_error`. Un controllo scritto nella forma tradizionale `if ($conn->connect_error)` non verrebbe quindi mai eseguito, e un database irraggiungibile produrrebbe una traccia di stack contenente host, utente e percorso del file. La connessione è perciò racchiusa in un `try/catch`: all'utente arriva un messaggio generico, il dettaglio tecnico finisce nel log del server tramite `error_log()`.

6. USABILITÀ E ACCESSIBILITÀ

L'accessibilità è stata trattata come requisito di progetto e non come rifinitura finale. Le verifiche sono state condotte con il **W3C Nu Markup Validation Service** per la correttezza formale e con **axe-core** (motore di analisi automatica delle regole WCAG) per il comportamento, su tutte le pagine e su due larghezze di viewport (1440 px e 390 px).

6.1 Comandi interattivi azionabili da tastiera

Il menu a scomparsa e la fisarmonica delle FAQ erano inizialmente <div> con un gestore onclick. Un <div> non è raggiungibile con il tasto Tab e non risponde a Invio o Barra spaziatrice: entrambi i comandi funzionavano solo con il mouse. L'aggiunta di role="button" e tabindex="0" al solo elemento del menu peggiorava anzi la situazione, perché faceva annunciare l'elemento come pulsante a uno screen reader senza renderlo effettivamente azionabile.

Entrambi sono stati riscritti come elementi <button type="button"> nativi. Un pulsante nativo è già nell'ordine di tabulazione, risponde a Invio e Barra spaziatrice senza codice aggiuntivo, viene annunciato correttamente e riceve gli stili di focus del sistema. Lo stato di apertura è esposto con aria-expanded, e aria-controls collega il comando al blocco che governa:

```
<button type="button" class="faq-trigger"
        aria-expanded="false" aria-controls="faq-a1">
  <span>Quanto tempo ci vuole per installare Foliarium?</span>
  <span class="faq-toggle" aria-hidden="true">+</span>
</button>
```

Il segno + che ruota in × è decorativo e duplicherebbe l'informazione già data da aria-expanded: è quindi marcato aria-hidden="true". Ogni domanda è inoltre racchiusa in un <h3>, così che la lista delle FAQ sia percorribile anche navigando per intestazioni.

Quando una risposta è chiusa, oltre a max-height: 0 le viene applicato visibility: hidden, con la transizione ritardata a fine animazione: il testo non resta leggibile agli screen reader mentre aria-expanded dichiara il blocco chiuso.

6.2 Contrasto cromatico

L'analisi automatica ha inizialmente rilevato **200 violazioni** del criterio WCAG 2.1 AA sul contrasto (1.4.3), distribuite su tutte e cinque le pagine. Le cause erano tre:

Causa	Rapporto misurato	Richiesto
Oro #B8821A come testo su fondo pergamena (tutte le etichette in maiuscoletto)	2,93:1	4,5:1
Testo avorio semitrasparente su fondo scuro con opacità troppo bassa (piè di pagina a .25)	2,07:1	4,5:1
Testo avorio sul pulsante oro	3,30:1	4,5:1

Gli interventi hanno preservato l'identità visiva invece di appiattire la palette:

- l'oro di marca --gold resta invariato per **filetti, bordi, pallini e sfondi**, dove il criterio sul contrasto del testo non si applica;
- per il **testo** sono state introdotte due varianti calibrate sul fondo: --gold-text: #845C10 sui fondi chiari (minimo 4,84:1) e --gold-on-dark: #C9942B sui fondi scuri (minimo 5,93:1);
- le opacità del testo chiaro su fondo scuro sono state portate al valore minimo che soddisfa il criterio (0,62 sull'inchiostro, 0,72 sul verde archivio);

- il pulsante oro ha ora testo color inchiostro (5,35:1), lo stesso trattamento già usato dalle altre etichette su fondo oro.

Al termine, le violazioni rilevate sono **0** su tutte le pagine e su entrambe le larghezze di viewport.

6.3 Struttura, landmark e collegamento di salto

Il contenuto di ogni pagina è racchiuso in `<main id="contenuto">`; con `<nav>` e `<footer>` questo garantisce che nessuna porzione di pagina resti fuori da un landmark, e permette agli utenti di screen reader di saltare direttamente al contenuto.

La prima tabulazione di ogni pagina incontra un **collegamento di salto**, nascosto fuori dallo schermo con `transform: translateY(-120%)` e reso visibile da `.skip-link:focus`. Senza di esso, un utente che naviga da tastiera dovrebbe attraversare l'intero menu su ogni pagina prima di raggiungere il testo.

La gerarchia delle intestazioni è continua su tutte le pagine: un solo `<h1>`, sezioni `<h2>`, blocchi `<h3>`, senza salti di livello.

6.4 Focus, movimento e modulo

- **Focus visibile.** Una regola `:focus-visible` unica applica un contorno oro a ogni elemento attivabile, con una variante più chiara sui fondi scuri. I campi del modulo avevano `outline: none` e affidavano il segnale di focus al solo colore del bordo: la dichiarazione è stata rimossa.
- **Movimento.** Le animazioni di ingresso e lo scorrimento morbido sono racchiusi in `@media (prefers-reduced-motion: no-preference)`; a chi ha richiesto meno animazioni nelle impostazioni di sistema il sito risponde senza transizioni.
- **Modulo.** Le etichette sono associate ai campi tramite `for/id`. Il campo del messaggio, obbligatorio lato server, porta ora anche l'attributo `required`, così l'errore viene segnalato dal browser prima dell'invio invece di costare un cambio di pagina. Nella scheda dei recapiti alcuni `<label>` erano usati come testo decorativo, senza alcun controllo da etichettare: sono stati sostituiti da `<span>`.

6.5 Validazione W3C

Tutte le pagine, **incluse entrambe le varianti della pagina di esito** (conferma ed errore), superano la validazione senza errori né avvisi. Le segnalazioni emerse durante lo sviluppo e come sono state risolte:

Segnalazione	Causa	Soluzione
aria-label on div	Attributo ARIA su un <code><div></code> privo di ruolo	Elemento riscritto come <code><button></code> nativo (§6.1)
Skipping heading level	<code><h4></code> sotto sezioni gestite con <code><h2></code>	Alberatura dei titoli ristrutturata su <code><h3></code>
Skipping heading level (piè di pagina)	I titoli di colonna erano <code><h3></code> ; nella pagina di esito, dove il titolo principale è un <code><h1></code> e non esistono sezioni <code><h2></code> , saltavano un livello	Titoli di colonna portati a <code><h2></code>
Stili inline	Attributi <code>style="..."</code> nei blocchi di layout	Regole centralizzate nel solo <code>style.css</code>

7. INSTALLAZIONE E COLLAUDO IN LOCALE

1. Avviare i moduli **Apache** e **MySQL/MariaDB** del proprio ambiente locale (XAMPP, MAMP o WAMP).
2. Copiare la cartella del progetto nella directory servita dal server (per esempio C:\xampp\htdocs\algora_site oppure C:\wamp64\www\algora_site).
3. Aprire **phpMyAdmin** (<http://localhost/phpmyadmin>) ed eseguire lo script crea_db.sql, che crea il database algora_db e la tabella contatti.
4. Verificare che i parametri in config-db.php corrispondano alla propria installazione (i valori predefiniti sono quelli di XAMPP/MAMP: utente root, password vuota).
5. Aprire http://localhost/algora_site/index.php.
6. Per collaudare la parte server-side, aprire http://localhost/algora_site/contatti.php, compilare il modulo e inviarlo. Va verificato che compaia la pagina di conferma e che la riga sia presente nella tabella contatti.

7.1 Casi di prova eseguiti

Caso	Esito atteso	Esito
Invio corretto	Pagina di conferma, riga inserita	Superato
Nome con apostrofo e & (Sant'Angelo & C.)	Caratteri corretti a video e nel database	Superato
Email formalmente non valida	Messaggio di errore specifico, nessun inserimento	Superato
Campi obbligatori vuoti	Messaggio di errore specifico, nessun inserimento	Superato
Apertura diretta di invia-contatto.php via URL	Redirect 303 al modulo	Superato
Stringa di SQL injection nel campo nome	Memorizzata come testo, tabella intatta	Superato
Database irraggiungibile	Messaggio generico all'utente, dettaglio nel log, nessun dato tecnico esposto	Superato

7.2 Verifica del passaggio a mobile-first

La riscrittura del foglio di stile non doveva cambiare il risultato a video. Per dimostrarlo sono state acquisite le schermate a pagina intera delle cinque pagine a 390, 768, 960, 961 e 1440 px prima e dopo l'intervento, e confrontate pixel per pixel:

- a 961 e 1440 px le immagini sono risultate **identiche**, a conferma che la disposizione desktop è invariata;
- a 390, 768 e 960 px l'unica differenza misurata riguarda l'altezza del piè di pagina, cioè esattamente la correzione descritta in §3.4; l'altezza di ogni altro elemento è rimasta invariata.

È stata inoltre verificata l'assenza di scorrimento orizzontale da 320 a 1920 px.

8. PUBBLICAZIONE ONLINE

Il sito è pensato per essere pubblicato sul dominio `www.algorastudio.it`. Su hosting condiviso la procedura è la seguente:

1. Caricare i file del progetto nella cartella pubblica del dominio (tipicamente `public_html` o `htdocs`).
 2. Creare il database dal pannello dell'hosting ed eseguirvi `crea_db.sql`.
 3. Aggiornare `config-db.php` con host, nome del database, utente e password forniti dal gestore del dominio, assegnando all'utente i soli privilegi necessari (INSERT e SELECT sulla tabella contatti).
 4. Verificare che `config-db.php` non sia raggiungibile dall'esterno e che non venga incluso nel versionamento.
-

9. LIMITI NOTI E SVILUPPI FUTURI

Per completezza si segnalano gli aspetti che un'evoluzione del progetto dovrebbe affrontare:

- **Contenuti multimediali.** Il sito è interamente testuale. Alcune schermate di Foliarium (ricerca fuzzy, albero delle proprietà) renderebbero più concreta la scheda prodotto e permetterebbero di mostrare la gestione dei testi alternativi.
- **Informativa privacy.** Il modulo raccoglie dati personali e li memorizza: i collegamenti "Privacy" e "Cookie" nel piè di pagina sono al momento segnaposto e vanno sostituiti da un'informativa reale, con relativa casella di consenso nel modulo.
- **Protezione del modulo.** Per un uso in produzione andrebbero aggiunti un token CSRF e una misura anti-spam.
- **Caratteri tipografici.** Ospitare localmente i due font eliminerebbe l'unica dipendenza esterna (§3.2).